

to five significant digits.

- (g) Given three variables **x**, **y**, **z** write a function to circularly shift their values to right. In other words if $x = 5$, $y = 8$, $z = 10$ after circular shift $y = 5$, $z = 8$, $x = 10$ after circular shift $y = 5$, $z = 8$ and $x = 10$. Call the function with variables **a**, **b**, **c** to circularly shift values.
- (h) Write a function to find the binary equivalent of a given decimal integer and display it.
- (i) If the lengths of the sides of a triangle are denoted by **a**, **b**, and **c**, then area of triangle is given by

$$area = \sqrt{S(S-a)(S-b)(S-c)}$$

where, $S = (a + b + c) / 2$

- (j) Write a function to compute the distance between two points and use it to develop another function that will compute the area of the triangle whose vertices are **A(x1, y1)**, **B(x2, y2)**, and **C(x3, y3)**. Use these functions to develop a function which returns a value 1 if the point **(x, y)** lies inside the triangle ABC, otherwise a value 0.
- (k) Write a function to compute the greatest common divisor given by Euclid's algorithm, exemplified for $J = 1980$, $K = 1617$ as follows:

$1980 / 1617 = 1$	$1980 - 1 * 1617 = 363$
$1617 / 363 = 4$	$1617 - 4 * 363 = 165$
$363 / 165 = 2$	$363 - 2 * 165 = 33$
$5 / 33 = 5$	$165 - 5 * 33 = 0$

Thus, the greatest common divisor is 33.

6 *Data Types Revisited*

- Integers, *long* and *short*
- Integers, *signed* and *unsigned*
- Chars, *signed* and *unsigned*
- Floats and Doubles
- A Few More Issues...
- Storage Classes in C
 - Automatic Storage Class
 - Register Storage Class
 - Static Storage Class
 - External Storage Class
 - Which to Use When
- Summary
- Exercise

As seen in the first chapter the primary data types could be of three varieties—**char**, **int**, and **float**. It may seem odd to many, how C programmers manage with such a tiny set of data types. Fact is, the C programmers aren't really deprived. They can derive many data types from these three types. In fact, the number of data types that can be derived in C, is in principle, unlimited. A C programmer can always invent whatever data type he needs.

Not only this, the primary data types themselves could be of several types. For example, a **char** could be an **unsigned char** or a **signed char**. Or an **int** could be a **short int** or a **long int**. Sufficiently confusing? Well, let us take a closer look at these variations of primary data types in this chapter.

To fully define a variable one needs to mention not only its type but also its storage class. In this chapter we would be exploring the different storage classes and their relevance in C programming.

Integers, *long* and *short*

We had seen earlier that the range of an Integer constant depends upon the compiler. For a 16-bit compiler like Turbo C or Turbo C++ the range is -32768 to 32767 . For a 32-bit compiler the range would be -2147483648 to $+2147483647$. Here a 16-bit compiler means that when it compiles a C program it generates machine language code that is targeted towards working on a 16-bit microprocessor like Intel 8086/8088. As against this, a 32-bit compiler like VC++ generates machine language code that is targeted towards a 32-bit microprocessor like Intel Pentium. Note that this does not mean that a program compiled using Turbo C would not work on 32-bit processor. It would run successfully but at that time the 32-bit processor would work as if it were a 16-bit processor. This happens because a 32-bit processor provides support for programs compiled using 16-bit compilers. If this backward compatibility support is not provided the 16-bit program

would not run on it. This is precisely what happens on the new Intel Itanium processors, which have withdrawn support for 16-bit code.

Remember that out of the two/four bytes used to store an integer, the highest bit (16th/32nd bit) is used to store the sign of the integer. This bit is 1 if the number is negative, and 0 if the number is positive.

C offers a variation of the integer data type that provides what are called **short** and **long** integer values. The intention of providing these variations is to provide integers with different ranges wherever possible. Though not a rule, **short** and **long** integers would usually occupy two and four bytes respectively. Each compiler can decide appropriate sizes depending on the operating system and hardware for which it is being written, subject to the following rules:

- (a) **shorts** are at least 2 bytes big
- (b) **longs** are at least 4 bytes big
- (c) **shorts** are never bigger than **ints**
- (d) **ints** are never bigger than **longs**

Figure 6.1 shows the sizes of different integers based upon the OS used.

Compiler	short	int	long
16-bit (Turbo C/C++)	2	2	4
32-bit (Visual C++)	2	4	4

Figure 6.1

long variables which hold **long** integers are declared using the keyword **long**, as in,

```
long int i ;  
long int abc ;
```

long integers cause the program to run a bit slower, but the range of values that we can use is expanded tremendously. The value of a **long** integer typically can vary from -2147483648 to +2147483647. More than this you should not need unless you are taking a world census.

If there are such things as **longs**, symmetry requires **shorts** as well—integers that need less space in memory and thus help speed up program execution. **short** integer variables are declared as,

```
short int j ;  
short int height ;
```

C allows the abbreviation of **short int** to **short** and of **long int** to **long**. So the declarations made above can be written as,

```
long i ;  
long abc ;  
short j ;  
short height ;
```

Naturally, most C programmers prefer this short-cut.

Sometimes we come across situations where the constant is small enough to be an **int**, but still we want to give it as much storage as a **long**. In such cases we add the suffix 'L' or 'l' at the end of the number, as in 23L.

Integers, *signed* and *unsigned*

Sometimes, we know in advance that the value stored in a given integer variable will always be positive—when it is being used to

only count things, for example. In such a case we can declare the variable to be **unsigned**, as in,

```
unsigned int num_students ;
```

With such a declaration, the range of permissible integer values (for a 16-bit OS) will shift from the range -32768 to +32767 to the range 0 to 65535. Thus, declaring an integer as **unsigned** almost doubles the size of the largest possible value that it can otherwise take. This so happens because on declaring the integer as **unsigned**, the left-most bit is now free and is not used to store the sign of the number. Note that an **unsigned** integer still occupies two bytes. This is how an **unsigned** integer can be declared:

```
unsigned int i ;  
unsigned i ;
```

Like an **unsigned int**, there also exists a **short unsigned int** and a **long unsigned int**. By default a **short int** is a **signed short int** and a **long int** is a **signed long int**.

Chars, *signed* and *unsigned*

Parallel to **signed** and **unsigned ints** (either **short** or **long**), similarly there also exist **signed** and **unsigned chars**, both occupying one byte each, but having different ranges. To begin with it might appear strange as to how a **char** can have a sign. Consider the statement

```
char ch = 'A' ;
```

Here what gets stored in **ch** is the binary equivalent of the ASCII value of 'A' (i.e. binary of 65). And if 65's binary can be stored, then -54's binary can also be stored (in a **signed char**).

A **signed char** is same as an ordinary **char** and has a range from -128 to +127; whereas, an **unsigned char** has a range from 0 to 255. Let us now see a program that illustrates this range:

```
main( )
{
    char ch = 291;
    printf ( "\n%d %c", ch, ch );
}
```

What output do you expect from this program? Possibly, 291 and the character corresponding to it. Well, not really. Surprised? The reason is that **ch** has been defined as a **char**, and a **char** cannot take a value bigger than +127. Hence when value of **ch** exceeds +127, an appropriate value from the other side of the range is picked up and stored in **ch**. This value in our case happens to be 35, hence 35 and its corresponding character #, gets printed out.

Here is another program that would make the concept clearer.

```
main( )
{
    char ch ;

    for ( ch = 0 ; ch <= 255 ; ch++ )
        printf ( "\n%d %c", ch, ch );
}
```

This program should output ASCII values and their corresponding characters. Well, No! This is an indefinite loop. The reason is that **ch** has been defined as a **char**, and a **char** cannot take values bigger than +127. Hence when value of **ch** is +127 and we perform **ch++** it becomes -128 instead of +128. -128 is less than 255 hence the condition is still satisfied. Here onwards **ch** would take values like -127, -126, -125, -2, -1, 0, +1, +2, ... +127, -128, -127, etc. Thus the value of **ch** would keep oscillating between -128 to +127, thereby ensuring that the loop never gets terminated. How do you

overcome this difficulty? Would declaring **ch** as an **unsigned char** solve the problem? Even this would not serve the purpose since when **ch** reaches a value 255, **ch++** would try to make it 256 which cannot be stored in an **unsigned char**. Thus the only alternative is to declare **ch** as an **int**. However, if we are bent upon writing the program using **unsigned char**, it can be done as shown below. The program is definitely less elegant, but workable all the same.

```
main( )
{
    unsigned char ch ;

    for ( ch = 0 ; ch <= 254 ; ch++ )
        printf ( "\n%d %c", ch, ch ) ;

    printf ( "\n%d %c", ch, ch ) ;
}
```

Floats and Doubles

A **float** occupies four bytes in memory and can range from -3.4e38 to +3.4e38. If this is insufficient then C offers a **double** data type that occupies 8 bytes in memory and has a range from -1.7e308 to +1.7e308. A variable of type **double** can be declared as,

```
double a, population ;
```

If the situation demands usage of real numbers that lie even beyond the range offered by **double** data type, then there exists a **long double** that can range from -1.7e4932 to +1.7e4932. A **long double** occupies 10 bytes in memory.

You would see that most of the times in C programming one is required to use either **chars** or **ints** and cases where **floats**, **doubles** or **long doubles** would be used are indeed rare.

Let us now write a program that puts to use all the data types that we have learnt in this chapter. Go through the following program carefully, which shows how to use these different data types. Note the format specifiers used to input and output these data types.

```
main( )
{
    char c ;
    unsigned char d ;
    int i ;
    unsigned int j ;
    short int k ;
    unsigned short int l ;
    long int m ;
    unsigned long int n ;
    float x ;
    double y ;
    long double z ;

    /* char */
    scanf ( "%c %c", &c, &d ) ;
    printf ( "%c %c", c, d ) ;

    /* int */
    scanf ( "%d %u", &i, &j ) ;
    printf ( "%d %u", i, j ) ;

    /* short int */
    scanf ( "%d %u", &k, &l ) ;
    printf ( "%d %u", k, l ) ;

    /* long int */
    scanf ( "%ld %lu", &m, &n ) ;
    printf ( "%ld %lu", m, n ) ;

    /* float, double, long double */
    scanf ( "%f %lf %Lf", &x, &y, &z ) ;
    printf ( "%f %lf %Lf", x, y, z ) ;
```

```
}

```

The essence of all the data types that we have learnt so far has been captured in Figure 6.2.

Data Type	Range	Bytes	Format
signed char	-128 to + 127	1	%c
unsigned char	0 to 255	1	%c
short signed int	-32768 to +32767	2	%d
short unsigned int	0 to 65535	2	%u
signed int	-32768 to +32767	2	%d
unsigned int	0 to 65535	2	%u
long signed int	-2147483648 to +2147483647	4	%ld
long unsigned int	0 to 4294967295	4	%lu
float	-3.4e38 to +3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to +1.7e4932	10	%Lf
Note: The sizes and ranges of int, short and long are compiler dependent. Sizes in this figure are for 16-bit compiler.			

Figure 6.2

A Few More Issues...

Having seen all the variations of the primary types let us take a look at some more related issues.

- (a) We saw earlier that size of an integer is compiler dependent. This is even true in case of **chars** and **floats**. Also, depending upon the microprocessor for which the compiler targets its code the accuracy of floating point calculations may change. For example, the result of 22.0/7.0 would be reported more

accurately by VC++ compiler as compared to TC/TC++ compilers. This is because TC/TC++ targets its compiled code to 8088/8086 (16-bit) microprocessors. Since these microprocessors do not offer floating point support, TC/TC++ performs all float operations using a software piece called Floating Point Emulator. This emulator has limitations and hence produces less accurate results. Also, this emulator becomes part of the EXE file, thereby increasing its size. In addition to this increased size there is a performance penalty since this bigger code would take more time to execute.

- (b) If you look at ranges of **chars** and **ints** there seems to be one extra number on the negative side. This is because a negative number is always stored as 2's compliment of its binary. For example, let us see how -128 is stored. Firstly, binary of 128 is calculated (10000000), then its 1's compliment is obtained (01111111). A 1's compliment is obtained by changing all 0s to 1s and 1s to 0s. Finally, 2's compliment of this number, i.e. 10000000, gets stored. A 2's compliment is obtained by adding 1 to the 1's compliment. Thus, for -128, 10000000 gets stored. This is an 8-bit number and it can be easily accommodated in a **char**. As against this, +128 cannot be stored in a **char** because its binary 010000000 (left-most 0 is for positive sign) is a 9-bit number. However +127 can be stored as its binary 01111111 turns out to be a 8-bit number.
- (c) What happens when we attempt to store +128 in a **char**? The first number on the negative side, i.e. -128 gets stored. This is because from the 9-bit binary of +128, 010000000, only the right-most 8 bits get stored. But when 10000000 is stored the left-most bit is 1 and it is treated as a sign bit. Thus the value of the number becomes -128 since it is indeed the binary of -128, as can be understood from (b) above. Similarly, you can verify that an attempt to store +129 in a **char** results in storing -127 in it. In general, if we exceed the range from positive side we end up on the negative side. Vice versa is

also true. If we exceed the range from negative side we end up on positive side.

Storage Classes in C

We have already said all that needs to be said about constants, but we are not finished with variables. To fully define a variable one needs to mention not only its ‘type’ but also its ‘storage class’. In other words, not only do all variables have a data type, they also have a ‘storage class’.

We have not mentioned storage classes yet, though we have written several programs in C. We were able to get away with this because storage classes have defaults. If we don’t specify the storage class of a variable in its declaration, the compiler will assume a storage class depending on the context in which the variable is used. Thus, variables have certain default storage classes.

From C compiler’s point of view, a variable name identifies some physical location within the computer where the string of bits representing the variable’s value is stored. There are basically two kinds of locations in a computer where such a value may be kept—Memory and CPU registers. It is the variable’s storage class that determines in which of these two locations the value is stored.

Moreover, a variable’s storage class tells us:

- (a) Where the variable would be stored.
- (b) What will be the initial value of the variable, if initial value is not specifically assigned.(i.e. the default initial value).
- (c) What is the scope of the variable; i.e. in which functions the value of the variable would be available.
- (d) What is the life of the variable; i.e. how long would the variable exist.

There are four storage classes in C:

- (a) Automatic storage class
- (b) Register storage class
- (c) Static storage class
- (d) External storage class

Let us examine these storage classes one by one.

Automatic Storage Class

The features of a variable defined to have an automatic storage class are as under:

Storage	– Memory.
Default initial value	– An unpredictable value, which is often called a garbage value.
Scope	– Local to the block in which the variable is defined.
Life	– Till the control remains within the block in which the variable is defined.

Following program shows how an automatic storage class variable is declared, and the fact that if the variable is not initialized it contains a garbage value.

```
main( )
{
    auto int i, j;
    printf( "\n%d %d", i, j );
}
```

The output of the above program could be...

1211 221

where, 1211 and 221 are garbage values of **i** and **j**. When you run this program you may get different values, since garbage values

are unpredictable. So always make it a point that you initialize the automatic variables properly, otherwise you are likely to get unexpected results. Note that the keyword for this storage class is **auto**, and not automatic.

Scope and life of an automatic variable is illustrated in the following program.

```
main( )
{
    auto int i = 1 ;
    {
        {
            printf ( "\n%d ", i );
        }
        printf ( "%d ", i );
    }
    printf ( "%d", i );
}
```

The output of the above program is:

1 1 1

This is because, all **printf()** statements occur within the outermost block (a block is all statements enclosed within a pair of braces) in which **i** has been defined. It means the scope of **i** is local to the block in which it is defined. The moment the control comes out of the block in which the variable is defined, the variable and its value is irretrievably lost. To catch my point, go through the following program.

```
main( )
{
    auto int i = 1 ;
    {
```

```
    auto int i = 2 ;
    {
        auto int i = 3 ;
        printf ( "\n%d ", i ) ;
    }
    printf ( "%d ", i ) ;
}
printf ( "%d", i ) ;
}
```

The output of the above program would be:

3 2 1

Note that the Compiler treats the three **i**'s as totally different variables, since they are defined in different blocks. Once the control comes out of the innermost block the variable **i** with value 3 is lost, and hence the **i** in the second **printf()** refers to **i** with value 2. Similarly, when the control comes out of the next innermost block, the third **printf()** refers to the **i** with value 1.

Understand the concept of life and scope of an automatic storage class variable thoroughly before proceeding with the next storage class.

Register Storage Class

The features of a variable defined to be of **register** storage class are as under:

Storage	- CPU registers.
Default initial value	- Garbage value.
Scope	- Local to the block in which the variable is defined.
Life	- Till the control remains within the block in which the variable is defined.

A value stored in a CPU register can always be accessed faster than the one that is stored in memory. Therefore, if a variable is used at many places in a program it is better to declare its storage class as **register**. A good example of frequently used variables is loop counters. We can name their storage class as **register**.

```
main( )
{
    register int i ;

    for ( i = 1 ; i <= 10 ; i++ )
        printf ( "\n%d", i ) ;
}
```

Here, even though we have declared the storage class of **i** as **register**, we cannot say for sure that the value of **i** would be stored in a CPU register. Why? Because the number of CPU registers are limited, and they may be busy doing some other task. What happens in such an event... the variable works as if its storage class is **auto**.

Not every type of variable can be stored in a CPU register.

For example, if the microprocessor has 16-bit registers then they cannot hold a **float** value or a **double** value, which require 4 and 8 bytes respectively. However, if you use the **register** storage class for a **float** or a **double** variable you won't get any error messages. All that would happen is the compiler would treat the variables to be of **auto** storage class.

Static Storage Class

The features of a variable defined to have a **static** storage class are as under:

- Storage – Memory.
- Default initial value – Zero.

Scope	– Local to the block in which the variable is defined.
Life	– Value of the variable persists between different function calls.

Compare the two programs and their output given in Figure 6.3 to understand the difference between the **automatic** and **static** storage classes.

<pre>main() { increment() ; increment() ; increment() ; } increment() { auto int i = 1 ; printf ("%d\n", i) ; i = i + 1 ; }</pre>	<pre>main() { increment() ; increment() ; increment() ; } increment() { static int i = 1 ; printf ("%d\n", i) ; i = i + 1 ; }</pre>
The output of the above programs would be:	
1	1
1	2
1	3

Figure 6.3

The programs above consist of two functions **main()** and **increment()**. The function **increment()** gets called from **main()** thrice. Each time it increments the value of **i** and prints it. The only difference in the two programs is that one uses an **auto** storage class for variable **i**, whereas the other uses **static** storage class.

Like **auto** variables, **static** variables are also local to the block in which they are declared. The difference between them is that **static** variables don't disappear when the function is no longer active. Their values persist. If the control comes back to the same function again the **static** variables have the same values they had last time around.

In the above example, when variable **i** is **auto**, each time **increment()** is called it is re-initialized to one. When the function terminates, **i** vanishes and its new value of 2 is lost. The result: no matter how many times we call **increment()**, **i** is initialized to 1 every time.

On the other hand, if **i** is **static**, it is initialized to 1 only once. It is never initialized again. During the first call to **increment()**, **i** is incremented to 2. Because **i** is static, this value persists. The next time **increment()** is called, **i** is not re-initialized to 1; on the contrary its old value 2 is still available. This current value of **i** (i.e. 2) gets printed and then **i = i + 1** adds 1 to **i** to get a value of 3. When **increment()** is called the third time, the current value of **i** (i.e. 3) gets printed and once again **i** is incremented. In short, if the storage class is **static** then the statement **static int i = 1** is executed only once, irrespective of how many times the same function is called.

Consider one more program.

```
main( )
{
    int *j;
    int *fun( );
    j = fun( );
    printf ( "\n%d", *j );
}

int *fun( )
{
```

```
    int k = 35 ;  
    return ( &k );  
}
```

Here we are returning an address of **k** from **fun()** and collecting it in **j**. Thus **j** becomes pointer to **k**. Then using this pointer we are printing the value of **k**. This correctly prints out 35. Now try calling any function (even **printf()**) immediately after the call to **fun()**. This time **printf()** prints a garbage value. Why does this happen? In the first case, when the control returned from **fun()** though **k** went dead it was still left on the stack. We then accessed this value using its address that was collected in **j**. But when we precede the call to **printf()** by a call to any other function, the stack is now changed, hence we get the garbage value. If we want to get the correct value each time then we must declare **k** as **static**. By doing this when the control returns from **fun()**, **k** would not die.

All this having been said, a word of advice—avoid using **static** variables unless you really need them. Because their values are kept in memory when the variables are not active, which means they take up space in memory that could otherwise be used by other variables.

External Storage Class

The features of a variable whose storage class has been defined as external are as follows:

Storage	–	Memory.
Default initial value	–	Zero.
Scope	–	Global.
Life	–	As long as the program's execution doesn't come to an end.

External variables differ from those we have already discussed in that their scope is global, not local. External variables are declared outside all functions, yet are available to all functions that care to use them. Here is an example to illustrate this fact.

```
int i ;
main( )
{
    printf ( "\ni = %d", i ) ;

    increment( ) ;
    increment( ) ;
    decrement( ) ;
    decrement( ) ;
}

increment( )
{
    i = i + 1 ;
    printf ( "\non incrementing i = %d", i ) ;
}

decrement( )
{
    i = i - 1 ;
    printf ( "\non decrementing i = %d", i ) ;
}
```

The output would be:

```
i = 0
on incrementing i = 1
on incrementing i = 2
on decrementing i = 1
on decrementing i = 0
```

As is obvious from the above output, the value of **i** is available to the functions **increment()** and **decrement()** since **i** has been declared outside all functions.

Look at the following program.

```
int x = 21 ;
main( )
{
    extern int y ;
    printf ( "\n%d %d", x, y ) ;
}
int y = 31 ;
```

Here, **x** and **y** both are global variables. Since both of them have been defined outside all the functions both enjoy external storage class. Note the difference between the following:

```
extern int y ;
int y = 31 ;
```

Here the first statement is a declaration, whereas the second is the definition. When we declare a variable no space is reserved for it, whereas, when we define it space gets reserved for it in memory. We had to declare **y** since it is being used in **printf()** before its definition is encountered. There was no need to declare **x** since its definition is done before its usage. Also remember that a variable can be declared several times but can be defined only once.

Another small issue—what will be the output of the following program?

```
int x = 10 ;
main( )
{
    int x = 20 ;

    printf ( "\n%d", x ) ;
```

```
        display( );  
    }  
    display( )  
    {  
        printf ( "\n%d", x );  
    }
```

Here **x** is defined at two places, once outside **main()** and once inside it. When the control reaches the **printf()** in **main()** which **x** gets printed? Whenever such a conflict arises, it's the local variable that gets preference over the global variable. Hence the **printf()** outputs 20. When **display()** is called and control reaches the **printf()** there is no such conflict. Hence this time the value of the global **x**, i.e. 10 gets printed.

One last thing—a **static** variable can also be declared outside all the functions. For all practical purposes it will be treated as an **extern** variable. However, the scope of this variable is limited to the same file in which it is declared. This means that the variable would not be available to any function that is defined in a file other than the file in which the variable is defined.

Which to Use When

Dennis Ritchie has made available to the C programmer a number of storage classes with varying features, believing that the programmer is in a best position to decide which one of these storage classes is to be used when. We can make a few ground rules for usage of different storage classes in different programming situations with a view to:

- (a) economise the memory space consumed by the variables
- (b) improve the speed of execution of the program

The rules are as under:

- Use **static** storage class only if you want the value of a variable to persist between different function calls.
- Use **register** storage class for only those variables that are being used very often in a program. Reason is, there are very few CPU registers at our disposal and many of them might be busy doing something else. Make careful utilization of the scarce resources. A typical application of **register** storage class is loop counters, which get used a number of times in a program.
- Use **extern** storage class for only those variables that are being used by almost all the functions in the program. This would avoid unnecessary passing of these variables as arguments when making a function call. Declaring all the variables as **extern** would amount to a lot of wastage of memory space because these variables would remain active throughout the life of the program.
- If you don't have any of the express needs mentioned above, then use the **auto** storage class. In fact most of the times we end up using the **auto** variables, because often it so happens that once we have used the variables in a function we don't mind losing them.

Summary

- (a) We can use different variations of the primary data types, namely **signed** and **unsigned char**, **long** and **short int**, **float**, **double** and **long double**. There are different format specifications for all these data types when they are used in **scanf()** and **printf()** functions.
- (b) The maximum value a variable can hold depends upon the number of bytes it occupies in memory.
- (c) By default all the variables are **signed**. We can declare a variable as **unsigned** to accommodate greater value without increasing the bytes occupied.

- (d) We can make use of proper storage classes like **auto**, **register**, **static** and **extern** to control four properties of the variable—storage, default initial value, scope and life.

Exercise

[A] What would be the output of the following programs:

- (a)

```
main( )
{
    int i ;
    for ( i = 0 ; i <= 50000 ; i++ )
        printf ( "\n%d", i ) ;
}
```
- (b)

```
main( )
{
    float a = 13.5 ;
    double b = 13.5 ;
    printf ( "\n%f %lf", a, b ) ;
}
```
- (c)

```
int i = 0 ;
main( )
{
    printf ( "\nmain's i = %d", i ) ;
    i++ ;
    val( ) ;
    printf ( "\nmain's i = %d", i ) ;
    val( ) ;
}
val( )
{
    i = 100 ;
    printf ( "\nval's i = %d", i ) ;
    i++ ;
}
```



```
(d) main( )
{
    int x, y, s = 2 ;
    s *= 3 ;
    y = f ( s ) ;
    x = g ( s ) ;
    printf ( "\n%d %d %d", s, y, x ) ;
}
int t = 8 ;
f ( int a )
{
    a += -5 ;
    t -= 4 ;
    return ( a + t ) ;
}
g ( int a )
{
    a = 1 ;
    t += a ;
    return ( a + t ) ;
}

(e) main( )
{
    static int count = 5 ;
    printf ( "\ncount = %d", count-- ) ;
    if ( count != 0 )
        main( ) ;
}

(f) main( )
{
    int i, j ;
    for ( i = 1 ; i < 5 ; i++ )
    {
        j = g ( i ) ;
        printf ( "\n%d", j ) ;
    }
```

```
    }  
    g ( int x )  
    {  
        static int v = 1 ;  
        int b = 3 ;  
        v += x ;  
        return ( v + x + b ) ;  
    }  
  
(g) float x = 4.5 ;  
    main( )  
    {  
        float y, float f ( float ) ;  
        x *= 2.0 ;  
        y = f ( x ) ;  
        printf ( "\n%f %f", x, y ) ;  
    }  
    float f ( float a )  
    {  
        a += 1.3 ;  
        x -= 4.5 ;  
        return ( a + x ) ;  
    }  
  
(h) main( )  
    {  
        func( ) ;  
        func( ) ;  
    }  
    func( )  
    {  
        auto int i = 0 ;  
        register int j = 0 ;  
        static int k = 0 ;  
        i++ ; j++ ; k++ ;  
        printf ( "\n %d %d %d", i, j, k ) ;  
    }
```

```
(i)  int x = 10 ;
      main( )
      {
          int x = 20 ;
          {
              int x = 30 ;
              printf ( "\n%d", x ) ;
          }
          printf ( "\n%d", x ) ;
      }
```

[B] Point out the errors, if any, in the following programs:

- ```
(a) main()
 {
 long num ;
 num = 2 ;
 printf ("\n%ld", num) ;
 }

(b) main()
 {
 char ch = 200 ;
 printf ("\n%d", ch) ;
 }

(c) main()
 {
 unsigned a = 25 ;
 long unsigned b = 25l ;
 printf ("\n%lu %u", a, b) ;
 }

(d) main()
 {
 long float a = 25.345e454 ;
 unsigned double b = 25 ;
 printf ("\n%lf %d", a, b) ;
 }
```

```
 }
(e) main()
 {
 float a = 25.345 ;
 float *b ;
 b = &a ;
 printf ("\n%f %u", a, b) ;
 }
(f) static int y ;
 main()
 {
 static int z ;
 printf ("%d %d", y, z) ;
 }
```

[C] State whether the following statements are True or False:

- (a) Storage for a register storage class variable is allocated each time the control reaches the block in which it is present.
- (b) An extern storage class variable is not available to the functions that precede its definition, unless the variable is explicitly declared in these functions.
- (c) The value of an automatic storage class variable persists between various function invocations.
- (d) If the CPU registers are not available, the register storage class variables are treated as static storage class variables.
- (e) The register storage class variables cannot hold float values.
- (f) If we try to use register storage class for a **float** variable the compiler will flash an error message.